

TEMA N° 1

I. INFORMACIÓN GENERAL:

Nombre del curso: FUNDAMENTOS DE PROGRAMACIÓN

Nombre del tema: Control de versiones y estructuras fundamentales

Título de la experiencia de laboratorio: “Introducción al control de versiones y fundamentos de programación en C#/Python”

II. OBJETIVO:

Que el estudiante **comprenda y aplique** las bases del control de versiones con Git y GitHub, además de implementar programas básicos en C# o Python utilizando estructuras secuenciales, condicionales y repetitivas, representando las soluciones mediante pseudocódigo y diagramas de flujo.

III. FUNDAMENTO TEÓRICO

- **Control de versiones:** Es un sistema que registra los cambios realizados en archivos de un proyecto, permitiendo volver a versiones anteriores y facilitar la colaboración. Git es un sistema distribuido, mientras que GitHub es una plataforma en línea que facilita la gestión remota de repositorios. Conceptos clave: commit, push, pull, branch, merge.
- **Fundamentos de programación (C#/Python):**
 - **Sintaxis básica:** Reglas del lenguaje que definen cómo escribir instrucciones válidas.
 - **Entrada/Salida (I/O):** Permite interactuar con el usuario a través de lectura y escritura en consola (`input()/print()` en Python, `Console.ReadLine()/Console.WriteLine()` en C#).
- **Estructuras de control:**
 - **Secuenciales:** Instrucciones ejecutadas en orden.
 - **Condicionales:** Permiten tomar decisiones (ej. if-else).
 - **Repetitivas (bucles):** Ejecutan bloques de código varias veces (for, while).

- **Pseudocódigo:** Representación textual de un algoritmo que describe pasos lógicos de manera estructurada y sencilla.
- **Diagramas de flujo:** Representación gráfica de algoritmos mediante símbolos estandarizados (inicio/fin, procesos, decisiones, entradas/salidas).

IV. HERRAMIENTAS, SIMULADORES Y/O RECURSOS

Lista de la plataforma, programa, simulador y/o recursos digitales necesarios para la experiencia

N°	Herramienta /Simuladores y/o Recurso Virtual	Uso o función específica	Acceso / Enlace
1	Git y GitHub	Control de versiones, colaboración y gestión de repositorios	https://github.com
2	Visual Studio Code / Visual Studio / PyCharm	Entorno de desarrollo integrado para programar en C# o Python	https://code.visualstudio.com
3	Draw.io o Lucidchart	Creación de diagramas de flujo de manera visual	https://app.diagrams.net

V. PROCEDIMIENTO

Parte A: Control de versiones con Git y GitHub

1. Instalar **Git** en la computadora.
2. Configurar usuario y correo con:

```
git config --global user.name "TuNombre"  
git config --global user.email "tuemail@example.com"
```

3. Crear un repositorio local:

```
git init
```

4. Crear un archivo de prueba hola.txt, agregarlo y hacer un commit.

```
git add hola.txt  
git commit -m "Primer commit"
```

5. Crear un repositorio en GitHub y vincularlo con el local.

```
git remote add origin <url-repositorio>  
git push -u origin master
```

6. Crear una rama llamada prueba, modificar el archivo y fusionar con la rama principal.

Parte B: Fundamentos de programación

1. Crear un programa “Hola mundo” en **Python** o **C#**.

- Python:

```
print("Hola mundo")
```

- C#:

```
Console.WriteLine("Hola mundo");
```

2. Implementar un programa que lea dos números e imprima su suma.

3. Diseñar un programa con estructura condicional que verifique si un número es par o impar.
4. Escribir un programa con estructura repetitiva que muestre los números del 1 al 10.

Parte C: Pseudocódigo y diagramas de flujo

1. Redactar en pseudocódigo el algoritmo del programa que suma dos números.
2. Dibujar el diagrama de flujo del programa que verifica si un número es par o impar.